

ML Defender (aRGus NDR): An Open-Source Embedded ML NIDS for Botnet and Anomalous Traffic Detection in Resource-Constrained Organizations

Alonso Isidoro Román
Independent Researcher, Extremadura, Spain
github.com/alonsoir/argus

Draft v22 — May 2026

Abstract

Ransomware and DDoS attacks disproportionately impact hospitals, schools, and small organizations that cannot afford enterprise security solutions. Existing open-source alternatives either rely on signature matching — missing novel variants — or require infrastructure beyond the reach of their target users. We present **ML Defender (aRGus NDR)**, an open-source network intrusion detection system built in C++20 that brings embedded machine learning inference to organizations with limited resources, deployable on commodity bare-metal hardware at approximately 150–200 USD fully dedicated.

ML Defender implements a six-component pipeline over eBPF/XDP packet capture, ZeroMQ transport, and Protocol Buffers serialization, terminating in a dual-score detection architecture combining a rule-based Fast Detector with an embedded Random Forest classifier. The *Maximum Threat Wins* policy selects the arithmetic maximum of both detector scores, using ML inference to suppress false positives generated by the heuristic layer.

We evaluate ML Defender against the CTU-13 Neris botnet dataset, achieving $F1 = 0.9985$, $Precision = 0.9969$, and $Recall = 1.0000$, with a false positive rate of 0.0002% (2 FP in 12,075 benign flows) — the two false positives identified as host-only adapter artifacts: an mDNS multicast frame (192.168.56.1 → 224.0.0.251) and a broadcast frame (192.168.56.1 → 192.168.56.255), both generated by the VirtualBox host-only NIC and absent from production network topologies. Whether these artifacts appear in bare-metal deployments remains unverified; bare-metal characterization is Future Work (§??). The Fast Detector alone produces a FPR of 6.61% on purely benign traffic (bigFlows); the ML Detector reduces real production blocks to zero on the same corpus, an approximately 500-fold reduction. Per-class inference latency ranges from 0.24 μ s (DDoS) to 1.06 μ s (Ransomware), enabling real-time detection on commodity hardware.

We further characterize the pipeline’s throughput ceiling under progressive load in a virtualized environment. Requesting up to 100 Mbps via `tcpreplay` against the VirtualBox NIC, the pipeline processes up to ~34–38 Mbps (the emulated NIC ceiling) with zero packet drops and zero errors across 2.37 million packets. At peak load, `ml-detector` consumes approximately 3.2 CPU cores; total pipeline RAM remains stable at ~1.28 GB with negligible drift (~18 MB over 8 minutes). The bottleneck is VirtualBox NIC emulation, not pipeline

logic. Post-replay drain behavior confirms that ZeroMQ queues absorb burst traffic correctly and drain asynchronously, validating the queue stability design. All throughput figures should be treated as conservative lower bounds; bare-metal characterization remains future work (§??).

These results demonstrate architectural feasibility under controlled replay conditions rather than universal detection capability. The evaluation covers a single botnet scenario from 2011 with synthetic-trained classifiers; generalizability to modern ransomware, DDoS variants, and encrypted C2 traffic has not been empirically established and is discussed in Section 5. Behavioral indicators associated with ransomware propagation and lateral-movement patterns are detected in the Neris scenario; direct evaluation against post-2020 ransomware families remains Future Work §??.

All experiments were conducted in a reproducible Vagrant/VirtualBox environment running on a consumer laptop, ensuring result accessibility and independent reproducibility.

This work was developed through a structured multi-model AI collaboration methodology — the *Consejo de Sabios* (Council of Wise Men), a multi-LLM peer review methodology — documented in Section ?? . *Test-Driven Hardening* (TDH) is proposed as a methodology for building security-critical distributed systems, emerging directly from this collaboration.

In a subsequent development phase (DAY 122), we integrated XGBoost 3.2.0 as a hot-swappable plugin classifier (ADR-026, Ed25519-signed) and conducted a rigorous temporal holdout evaluation on CIC-IDS-2017, using Tuesday, Thursday, and Friday as training data and Wednesday as a sealed blind test set. The in-distribution model achieves Precision= 0.9945 and Recall= 0.9818 on the validation split. However, evaluation on the Wednesday holdout reveals a structural covariate shift: attack types present exclusively in Wednesday (DoS Hulk, GoldenEye, Slowloris) are absent from all training days, making generalization mathematically impossible regardless of threshold. We document this as a property of the dataset’s design — not of the algorithm — and demonstrate empirically that no threshold simultaneously satisfies $\text{Precision} \geq 0.99$ and $\text{Recall} \geq 0.95$ on the out-of-distribution split. This finding corroborates and quantifies the thesis of Sommer and Paxson [14]: static models trained on academic benchmarks are structurally insufficient for production NDR. The architectural response — an Adversarial Capture-Retrain Loop in which a generative red team agent produces attack traffic, the pipeline captures it, and the classifier is retrained on real flows — is introduced as future work (§??).

We further report the first empirical comparison of ADR-029 Variant A (eBPF/XDP) and Variant B (libpcap) on x86-64 (DAY 145). Under VirtualBox virtio emulation, Variant B achieves approximately twice the throughput of Variant A at 50 and 100 Mbps nominal rates (~19 Mbps vs ~10 Mbps), an inversion of the expected ordering on physical hardware. Both variants process the complete Neris corpus without pipeline errors, confirming functional equivalence at the detection layer. The inversion is attributed to the absence of a native XDP driver path in the virtio NIC, causing eBPF to fall back to generic SKB mode. This result provides direct empirical motivation for bare-metal hardware acquisition (§??).

We further report the first *direct experimental comparison* of aRGus NDR and Suricata 6.0.10 under identical controlled conditions (DAY 146): same hardware, same dataset (CTU-13 Neris), same network topology, same replay speeds (§3.13). Suricata with 50,010 current ET Open rules generates **zero alerts** on the 2011 Neris corpus; aRGus achieves F1= 0.9985 and Recall= 1.0000 on the same traffic. This result provides direct empirical evidence that signature-based detection requires prior knowledge of the threat, while ML-based behavioral detection does not — corroborating the thesis of Sommer and Paxson [14] with reproducible experimental data. The zero-alert result is not a failure of Suricata:

the ET Open ruleset evolves continuously and rules for 15-year-old threats are retired as new rules emerge. We extend this head-to-head to a *three-paradigm comparison* by adding Zeek 8.1.2 with default policy scripts (DAY 147, §3.14): Zeek generates 14 correct detections (`SSL::Invalid_Server_Cert`, Precision= 1.0000, F1= 0.0424) while fully observing the botnet behavioral profile in structured logs (`weird.log`, `http.log`, `smtp.log`) without generating further alerts under default configuration — placing scripted behavioral detection precisely between signature-based and ML-based approaches on the Recall axis (F1= 0.042 vs F1= 0.9985).

ML Defender is released under the MIT license.

Contents

1	Introduction	4
2	Background and Related Work	5
3	Evaluation	6
3.1	Experimental Setup	6
3.2	Dataset	6
3.3	Detection Performance	7
3.4	False Positive Identification and Confusion Matrix	7
3.5	Inference Latency	7
3.6	Ablation Study	7
3.7	Interpretation and Comparison with State of the Art	8
3.8	Ransomware Detection: Scope and Validation	8
3.9	Throughput Stress Test and Resource Utilization	9
3.10	XGBoost Level-1 Classifier: In-Distribution Performance	9
3.11	XGBoost Level-1: Wednesday Out-of-Distribution Evaluation	9
3.12	ADR-029: Capture Backend Comparison — Variant A (eBPF) vs Variant B (libpcap)	9
3.13	Direct Experimental Comparison: aRGus NDR vs Suricata 6.0.10 on CTU-13 Neris	10
3.14	Three Paradigms: Signature, Scripted Behavioral, and ML Behavioral Detection	13
4	Performance Model and Throughput Analysis	15
5	Limitations	15
6	Future Work	15
7	Conclusion	15
8	Reproducibility and Artifact Availability	17

1 Introduction

The threat is not abstract. On March 5, 2023, the Hospital Clínic de Barcelona suffered a ransomware attack that paralyzed its emergency services, laboratory, and pharmacy, encrypted 4.5 TB of patient data, and prompted a ransom demand of 4.5 million USD — which the institution refused to pay [8]. That same day, the author of this work was leaving a regional hospital in Extremadura after receiving treatment for porphyria — a rare metabolic disease that his local hospital had diagnosed and managed. Watching the news, the thought was immediate and personal: *if it can happen to a major teaching hospital, it can happen to mine.*

This was not the first time that the author had witnessed the human cost of ransomware firsthand. Years earlier, the collapse of a close friend’s small business had been caused by an encryption attack that locked every document and file in his office. The author lacked the technical skills to help. That failure left a debt.

These two events — the friend’s destroyed business, the hospital on television — converged into a single design constraint: the solution had to be open-source, it had to run on commodity hardware, and it had to act in real time. Not forensics. Not post-mortem analysis. Interception.

The analogy that emerged from this motivation is biological. Network traffic behaves like a circulatory system: datagrams are the bloodstream, and malicious flows are pathogens. A useful defense must classify traffic continuously and in real time — capturing the source IP of the attacker and blocking it at the firewall before damage propagates. This framing shaped every architectural decision in ML Defender.

This work therefore addresses the following research question: *can an embedded machine-learning network detection and response system running entirely on commodity hardware provide meaningful real-time protection for organizations lacking enterprise security infrastructure?*

A note on terminology. ML Defender (aRGus NDR) implements *Network Detection and Response* (NDR) capabilities: real-time capture, ML-based classification, and automated blocking at the network layer. Future work toward full *Endpoint Detection and Response* (EDR) functionality via a planned lightweight endpoint agent (§6, FEAT-EDR-1) will be reflected in a corresponding rename when that capability is delivered.

The gap. Existing open-source NIDS — most notably Snort and Suricata [10] — operate primarily on signature matching and rule-based heuristics. While effective against known attack patterns, they offer limited resilience against novel ransomware variants and volumetric DDoS campaigns. Machine learning-based NIDS have been extensively studied [5, 11], but the gap between research prototype and deployable system remains wide: most published systems are evaluated on benchmark datasets without accompanying implementation, require GPU infrastructure, or assume dedicated security operations teams. Healthcare retained its position as the most targeted sector in 2025, accounting for 22% of disclosed ransomware attacks, with an average breach cost of 7.42M USD [4, 7].

Our approach. ML Defender (aRGus NDR) addresses this gap with three design principles: (1) *embedded inference* — all ML classification runs in-process at sub-microsecond latency;

(2) *commodity deployment* — the full six-component pipeline runs on bare-metal Linux hardware costing approximately 150–200 USD; (3) *active response* — upon detection, the attacker’s IP is blocked immediately at the firewall via ipset/iptables.

Beyond the technical system, this paper documents a second contribution: the *Consejo de Sabios* methodology for human-AI collaborative engineering.

2 Background and Related Work

Signature-based NIDS. Snort [12] and Suricata [10] apply rule-based pattern matching against packet payloads and flow metadata. Their fundamental limitation is reactive: detection requires a prior known signature, a window that historically ranges from hours to weeks.

ML-based NIDS. Anderson & McGrew [1] demonstrated the feasibility of contextual flow data for encrypted malware classification. Mirsky et al. [9] proposed Kitsune, an ensemble of autoencoders for unsupervised anomaly detection operating on flow statistics. The CTU-13 dataset [6], which ML Defender uses for evaluation, has become a standard benchmark for botnet traffic detection. Despite this body of work, a persistent gap exists between academic prototype and production system. Sommer and Paxson [14] identified fundamental reasons for this gap: the closed-world assumption of academic datasets, high base-rate asymmetry, and the absence of diversity in training distributions. ML Defender’s DAY 122 evaluation provides new empirical evidence for this thesis (§3.11).

Flow-based ML NIDS. ML Defender operates exclusively at the flow level, extracting 28 behavioral features per flow without examining payload content — ensuring that payload encryption does not compromise detection capability.

eBPF and XDP-based network monitoring. Recent work has explored eBPF/XDP as a foundation for high-performance network security. ML Defender follows the XDP-capture-plus-userspace-inference architecture: XDP provides zero-copy packet capture before the kernel networking stack, while all ML inference executes in userspace at sub-microsecond latency. To our knowledge, open-source projects combining eBPF capture with userspace ML inference remain rare; most production-grade alternatives either offload inference to the cloud or require GPU hardware.

Embedded inference. Commercial EDR vendors (CrowdStrike, SentinelOne, Carbon Black) implement proprietary embedded ML pipelines, but neither their architectures nor their training data are publicly available. ML Defender demonstrates that embedded Random Forest inference achieves sub-microsecond latency without requiring GPU acceleration or cloud offload.

Human-AI collaborative engineering. Structured methodologies for multi-model peer review have not, to our knowledge, been formally documented in the systems security literature. Section ?? describes the *Consejo de Sabios* methodology.

AI-native security reasoning and the evolving threat landscape. This paper was written and submitted in April 2026, concurrent with the announcement of Anthropic’s Project Glasswing [2], which demonstrated that AI models can autonomously identify and chain kernel-level vulnerabilities — including local privilege escalation to root in Linux — at a scale and speed previously requiring specialized human expertise. These results represent a shift in the threat landscape: AI-augmented offensive capabilities are no longer theoretical. This directly motivates the explicit kernel security boundary axiom in §??: aRGus NDR assumes the kernel as a potentially compromised boundary and shifts its trust anchor to verifiable network behavioral patterns. The network remains an observable chokepoint even when the host is not. The hardened deployment variants ADR-030 (AppArmor) and ADR-031 (seL4) documented in §?? are a direct architectural response to this trajectory.

3 Evaluation

3.1 Experimental Setup

Table 1: Experimental hardware and software specifications.

Parameter	Value
Host machine	Apple MacBook Pro (Intel Core i9, 2.5 GHz)
Host RAM	32 GB DDR4
Guest OS	Ubuntu 24.04 LTS (Linux kernel 6.x)
Virtualization	VirtualBox 7.2, bridged NIC mode
VM vCPUs	4 (estimated ~ 2.4 GHz under VirtualBox)
VM RAM	4 GB (8 GB allocated for stress tests)
Network adapter	Paravirtualized (VirtIO)
Virt. overhead	Estimated 5–15% vs bare-metal
Pipeline config.	Full 6-component, dual-VM

Methodological note on metrics. `calculate_f1_neris.py` measures **Fast Detector** alerts ([FAST ALERT] in sniffer.log) against CTU-13 Neris ground truth IPs. The ML Detector’s `attacks=N` statistic reports flows exceeding production thresholds and triggering firewall blocks. Both are reported; F1=0.9985 is the primary system detection metric.

3.2 Dataset

CTU-13 Neris. 19,135 total flows; **646 malicious flows** (host 147.32.84.165) constitute the TP ground truth. Captured 2011. Garcia et al. [6] document that the Neris botnet exhibits C2 communication patterns concentrated in a behavioral subset — specifically, flows matching IRC-based C2 channel signatures (connection bursts, SMB lateral movement, DNS query anomalies). The remaining flows represent background traffic on the infected host that do not exhibit malicious behavioral signatures and are therefore not counted as ground-truth positives. This distinction aligns with standard practice in NIDS evaluation [9, 13].

BigFlows. A CTU-13 derived dataset containing 40,467 flows and 791,615 packets, used for high-throughput replay experiments. 38,064 flows were processed by ml-detector. Treated conservatively as *probable benign* — no official ground truth exists for this CTU-13 scenario.

Synthetic training data. All classifiers trained exclusively on synthetic data; CTU-13 Neris held out entirely for evaluation.

3.3 Detection Performance

Table 2: Detection performance — Fast Detector pipeline on CTU-13 Neris.

Metric	Value
F1 Score	0.9985
Precision	0.9969
Recall	1.0000
True Positives	646
False Positives	2
False Negatives	0
True Negatives	12,075
Total events processed	12,723
FPR (Neris evaluation)	0.0002%
ML Detector high-conf. attacks	12
FPR Fast Detector (bigFlows)	6.61%
FP reduction (Fast → ML)	≈500-fold

F1=0.9985 stable across replay runs at total_events = 12,563 / 12,605 / 12,723 / 13,930.

3.4 False Positive Identification and Confusion Matrix

Table 3: False positive identification.

FP	Source	Destination	Classification
FP-1	192.168.56.1	224.0.0.251	mDNS multicast (VirtualBox host-only)
FP-2	192.168.56.1	192.168.56.255	Broadcast (VirtualBox host-only)

Both are VirtualBox host-only adapter artifacts. Their absence in bare-metal deployments is a reasonable expectation, but has not been empirically verified (§??).

Confusion matrix.

$$\text{Precision} = \frac{646}{648} = 0.9969, \quad \text{Recall} = \frac{646}{646} = 1.0000, \quad \text{F1} = 0.9985$$

3.5 Inference Latency

3.6 Ablation Study

Config A — Fast Detector Only: 2,517 FP on 38,064 bigFlows (FPR=6.61%), Recall=1.0000 on Neris.

Table 4: Per-class inference latency.

Classifier	Implementation	Latency
DDoS	Embedded C++20	0.24 μ s
Traffic	Embedded C++20	0.37 μ s
Internal	ONNX Runtime	0.33 μ s
Ransomware	Embedded C++20	1.06 μ s

Config B — ML Detector Only (inferred): 5 detections on 38,064 bigFlows, all below production thresholds (max confidence 68.97%) — 0 real blocks. Neris: attacks=12.

Config C — Dual (deployed): F1=0.9985, FPR=0.0002%. Only Config C fully validated end-to-end.

3.7 Interpretation and Comparison with State of the Art

F1=0.9985 and Recall=1.0000 reflect the statistical separability of the Neris botnet scenario, not a claim about arbitrary threats. The synthetic training successfully captured the Neris behavioral signature; generalization to other families is an open empirical question.

Table 5: Comparison with representative NIDS approaches. The Suricata 6.0.10 row reports an *empirically measured* result on CTU-13 Neris (DAY 146, §3.13); all other rows are indicative cross-paper comparisons. [§] Suricata 6.0.10 with 50,010 ET Open rules (May 2026) generates zero alerts on CTU-13 Neris 2011. This is not a system failure — ET Open rules for this 15-year-old botnet family are no longer active in the current ruleset. *Caution: non-Suricata rows are indicative only — different datasets, operating points, and conditions.*

System	Dataset	F1	Latency	HW	Block	Notes
ML Defender	CTU-13 Neris	0.9985	0.24–1.06 μ s	x86/ARM	Yes	Synthetic, dual-score
Kitsune [9]	Custom	~0.982	~50 μ s	x86	No	Autoencoder ensemble
Suricata 6.0.10 [§]	CTU-13 Neris	0.000	~10 μ s	x86	Yes	0 alerts (no rule for Neris 2011)
RF [13]	CIC-IDS2017	~0.943	~200 μ s	x86	No	Batch, not embedded
Snort	N/A	N/A	1–5 ms	x86	Yes	Rule-based, no ML

3.8 Ransomware Detection: Scope and Validation

What is validated. Behavioral indicators associated with ransomware propagation and lateral-movement patterns are detected in the 2011 Neris botnet scenario: SMB lateral movement (ports 445/139 burst), high-rate connection attempts, DNS query anomalies.

What is not validated. Direct evaluation against post-2020 ransomware families (LockBit, BlackCat/ALPHV, Cl0p) has not been performed. Modern ransomware uses encrypted C2, adaptive beaconing with jitter, staged delivery — potentially different statistical signatures.

Future work. Direct evaluation against modern ransomware captures is the highest-priority evaluation expansion (§??), with Sebastian Garcia (CTU Prague) as potential collaborator.

Table 6: Stress test replay results — progressive Mbps escalation (DAY 87).

Run	Dataset	Req.	Actual	PPS	Packets	Failed	Errors
1	Neris	10 Mbps	9.60 Mbps	~3,700	320,524	2,630 [†]	0
2	Neris	25 Mbps	11.28 Mbps	~4,300	320,524	2,630 [†]	0
3	bigFlows	50 Mbps	34.68 Mbps	9–10K	791,615	0	0
4	bigFlows	100 Mbps (×3)	33.16 Mbps	9–11K	2,374,845	0	0

Pre-existing pcap artifact — identical count at 10 and 25 Mbps, confirming origin in pcap structure, not pipeline saturation.

Table 7: Resource utilization during 100 Mbps bigFlows replay (DAY 87).

Component	CPU%	RES	MEM%	Notes
ml-detector	~315–320%	164 MB	2.0%	3.2 cores; multi-threaded
sniffer	~88–108%	55–57 MB	0.7%	XDP + ring consumer
firewall-acl	~10–16%	101 MB	1.2%	Stable throughout
rag-ingester	~12–20%	36–39 MB	0.5%	FAISS indexing
etcd-server	~0.2%	10 MB	0.1%	Minimal
rag-security	idle	1.2 GB VIRT	15.0% [‡]	TinyLlama loaded
Total	~65–73%	~1,278 MB	—	~23–30% idle

rag-security’s 1.2 GB virtual footprint is the TinyLlama model mapped at startup.

3.9 Throughput Stress Test and Resource Utilization

The throughput ceiling is VirtualBox NIC emulation (~33–38 Mbps), not pipeline logic. Zero packet drops, zero deserialization errors across all 2,374,845 packets. Total pipeline RAM stable at ~1.28 GB (±18 MB drift) throughout the 8-minute test.

3.10 XGBoost Level-1 Classifier: In-Distribution Performance

3.11 XGBoost Level-1: Wednesday Out-of-Distribution Evaluation

3.12 ADR-029: Capture Backend Comparison — Variant A (eBPF) vs Variant B (libpcap)

ADR-029 defines three capture backend variants for the aRGus sniffer: Variant A (eBPF/XDP, reference), Variant B (libpcap userspace), and Variant C (seL4/libpcap, research-only). This section reports the first empirical comparison of Variants A and B on x86-64 under identical conditions (DAY 145).

Experimental protocol. Both variants were evaluated against the full CTU-13 Neris pcap (320,524 packets, 19,135 flows) replayed from a dedicated client VM via `tcpreplay` at nominal rates of 10, 50, and 100 Mbps. The pipeline ran in its standard 6-component configuration; only the sniffer binary was swapped between runs. The mutual-exclusion invariant — Variant A and Variant B never active simultaneously — was enforced mechanically via a Makefile guard (`CHECK_SNIFFER_MUTEX`).

Table 8: ADR-029 Variant A (eBPF) vs Variant B (libpcap) — CTU-13 Neris replay on x86-64 VirtualBox (DAY 145). Both variants: 320,524 packets sent, 2,630 failed (pre-existing pcap MTU artifact, `errno=90`), 0 pipeline errors.

Variant	Target	Actual (Mbps)	PPS	Duration (s)	exit
A – eBPF	10 Mbps	8.86	8,040	39.86	0
A – eBPF	50 Mbps	9.78	8,867	36.14	0
A – eBPF	100 Mbps	10.12	9,178	34.92	0
B – libpcap	10 Mbps	9.99	9,064	35.36	0
B – libpcap	50 Mbps	19.43	17,614	18.19	0
B – libpcap	100 Mbps	18.82	17,066	18.78	0

Result. Variant B (libpcap) achieves approximately twice the throughput of Variant A (eBPF) at 50 and 100 Mbps nominal rates (~ 19 Mbps vs ~ 10 Mbps). Variant A plateaus at ~ 10 Mbps regardless of the requested rate.

Interpretation: VirtualBox virtio as the confounding factor. This result is *counter-intuitive* with respect to the expected performance ordering. On physical hardware with a NIC supporting native XDP, eBPF/XDP provides zero-copy packet delivery before the kernel networking stack and is expected to outperform libpcap. In the VirtualBox environment, the paravirtualized virtio NIC does not expose a native XDP driver path, causing eBPF to fall back to generic SKB mode.

Scientific value. This inversion constitutes direct empirical evidence that the relative performance of eBPF/XDP vs libpcap is critically dependent on the hardware and driver stack. The VirtualBox result establishes a reproducible baseline; the expected inversion on physical hardware with XDP-native NIC drivers (Intel ixgbe, Mellanox mlx5) remains to be measured (§??).

Both variants processed the complete 320,524-packet Neris corpus without pipeline errors, confirming functional equivalence at the detection layer. The 2,630 failed packets are a pre-existing pcap artifact (`errno=90 EMSGSIZE`) present at identical count in both runs, confirming origin in the pcap structure rather than in either capture backend.

3.13 Direct Experimental Comparison: aRGus NDR vs Suricata 6.0.10 on CTU-13 Neris

To complement the indicative comparison in Table 5, we conducted a direct head-to-head experiment between aRGus NDR and Suricata 6.0.10 under *identical controlled conditions* (DAY 146). This is, to our knowledge, the first published direct experimental comparison of an embedded ML-based NDR system and a production signature-based IDS on the same dataset, hardware, and network topology.

Experimental protocol. Both systems were evaluated on the full CTU-13 Neris pcap (320,524 packets, 19,135 flows, ground truth: 147.32.84.165, 646 malicious flows, 12,075 be-

nign flows) replayed from a dedicated client VM via `tcpreplay` at nominal rates of 10, 50, and 100 Mbps. The Suricata experiment used an identical VM specification to the aRGus evaluation: `debian/bookworm64 v12.20240905.1`, 8,192 MB RAM, 6 vCPUs, VirtIO NIC, VirtualBox 7.2. Suricata was evaluated with the Emerging Threats (ET) Open ruleset as distributed by `suricata-update` on 9 May 2026 (50,010 active rules). No custom tuning was applied; the evaluation reflects out-of-the-box detection capability with the current community ruleset.

The network topology replicates the aRGus evaluation exactly: a dedicated client VM sends the CTU-13 Neris pcap over an internal VirtualBox network (`suricata_experiment_lan`) to the IDS VM, which listens in passive mode on `eth2` (promiscuous mode, VirtIO). Makefile targets `make experiment-suricata-run` and `parse_results.py` provide reproducible one-command execution (§8).

Table 9: Direct detection comparison: aRGus NDR vs Suricata 6.0.10 — CTU-13 Neris replay, x86-64 VirtualBox (DAY 146). Identical hardware, dataset, topology, and speeds. Ground truth: 147.32.84.165 (646 malicious flows, 12,075 benign flows).

System	Detection basis	TP	FP	FN	F1	Recall
aRGus NDR	ML behavioral (synthetic)	646	2	0	0.9985	1.0000
Suricata 6.0.10	ET Open rules (May 2026)	0	0	646	0.0000	0.0000

Table 10: Suricata 6.0.10 throughput on CTU-13 Neris replay (DAY 146). All runs: 320,524 packets sent, 2,630 failed (pre-existing pcap MTU artifact, `errno=90`, identical to aRGus runs). 0 alerts generated.

Target	Actual (Mbps)	Alerts	exit
10 Mbps	9.99	0	0
50 Mbps	19.43	0	0
100 Mbps	18.82	0	0

Result: Suricata generates zero alerts on CTU-13 Neris 2011. Suricata 6.0.10 with 50,010 current ET Open rules produces zero detections across all three replay speeds. The `eve.json` output contains only `stats` events; no `alert` events are generated for any flow from the ground truth IP (147.32.84.165) or from any other host in the Neris corpus. The `detect.engines[0].rules_loaded` counter confirms 50,010 rules were active during each run.

Interpretation: ruleset evolution, not system failure. This result does not indicate a deficiency in Suricata as a system. It reflects a fundamental property of signature-based detection: *a rule must exist to produce an alert*. The ET Open ruleset is a living corpus that evolves continuously — rules for obsolete or inactive threat families are retired as new signatures emerge. The Neris botnet was captured in 2011; its IRC-based C2 channel, SMB lateral movement patterns, and DNS anomaly indicators are 15 years old. The ET Open maintainers have retired the specific signatures that would have matched this 2011 traffic, replacing them with coverage for contemporary threats.

The corollary is equally important: aRGus NDR achieves Recall= 1.0000 and F1= 0.9985 on the same 2011 traffic *without any prior knowledge of the Neris family*. The behavioral features extracted by the Fast Detector — external IP velocity, SMB connection diversity, TCP RST ratio, port scan diversity — are independent of the threat’s age, name, or ruleset coverage. A botnet exhibiting IRC C2 communication, SMB lateral movement, and DNS anomalies is detectable through its behavioral footprint whether it was captured in 2011 or 2026.

Scientific significance. This experiment provides direct empirical evidence for the core thesis of this paper and of Sommer and Paxson [14]: *signature-based detection requires prior knowledge of the threat; ML-based behavioral detection does not*. The experimental conditions are controlled, the dataset is public, and the comparison is direct rather than indicative. Table 5 has been updated to reflect the empirically measured Suricata result on this dataset.

Methodological note on fairness. The comparison is fair in hardware, topology, dataset, and replay conditions. One asymmetry favors aRGus NDR: its classifiers were trained on synthetic data *informed by CTU-13 flow statistics*, providing indirect exposure to the Neris behavioral profile. A fully blind evaluation would require training on a dataset with no statistical overlap with CTU-13. This limitation is acknowledged in §5 (10.3). The behavioral separability of the Neris scenario means this indirect advantage may be modest, but it cannot be ruled out without an independent blind evaluation.

Historical rulesets: DAY 147 search and findings. A complete characterization of the signature-vs-behavior gap would require repeating the Suricata experiment with the ET Open ruleset *from August 2011*, to distinguish “the rule existed and was later retired” (signature aging) from “the rule was never written” (coverage gap from inception).

A systematic search was conducted in DAY 147 across three independent sources: (1) the Wayback Machine CDX API — binary `.tar.gz` archives are not indexed as retrievable snapshots; (2) the EmergingThreats GitHub organization — no historical rule repositories are publicly available; (3) bundled rulesets distributed with SecurityOnion and alienfault/ossim — creation timestamps are unverifiable for August 2011. No public archive of the ET Open ruleset from August 2011 was located.

The search yielded an additional finding with direct scientific relevance: the official CTU-13 Neris dataset documentation [6] records the bot’s C2 channel as *HTTP-based*, not IRC. The scenario 42 README states explicitly: “*The bot sent spam, connected to an HTTP CC, and use HTTP to do some ClickFraud.*” This implies that IRC-specific botnet signatures — the most prevalent category of 2011 ET Open rules targeting Neris-family malware — would not have matched this specific capture regardless of ruleset vintage. The behavioral paradigm gap between ML-based and signature-based detection is therefore deeper than the signature aging hypothesis alone suggests: even a contemporaneous ruleset would have required prior knowledge of Neris’s specific HTTP C2 patterns.

The signature aging phenomenon is independently documented in the academic literature. Asad and Gashi [3] conducted a perspective-retrospective analysis of Snort and Suricata ET Open rules over a four-year window (2017–2020), finding that detection performance does not evolve

linearly with ruleset updates — a quantitative confirmation that signatures for historical threat families are retired as new coverage emerges [3].

The inability to locate public historical ruleset archives is itself a finding: it motivates the security community to maintain versioned, time-stamped public archives of open-source IDS rulesets — analogous to software package repositories with locked dependency snapshots — to enable reproducible longitudinal evaluation. This infrastructure gap is noted as a community recommendation.

3.14 Three Paradigms: Signature, Scripted Behavioral, and ML Behavioral Detection

To complete the paradigm comparison initiated in §3.13, we extended the direct experimental comparison to include Zeek 8.1.2 with default policy scripts (DAY 147). The three systems represent distinct detection philosophies evaluated under identical conditions on the same corpus.

Experimental protocol. Zeek 8.1.2 was provisioned on a dedicated VM with the same specification as the Suricata and aRGus evaluations: `debian/bookworm64 v12.20240905.1`, 8,192 MB RAM, 6 vCPUs, VirtIO NIC, VirtualBox 7.2. Zeek was evaluated in *offline mode* (`zeek -r neris.pcap local`), reading the CTU-13 Neris pcap directly without live capture. This mode processes 100% of the 320,524 packets deterministically, eliminating throughput-dependent packet loss as a confounding variable. The experiment executed three times at nominal 10, 50, and 100 Mbps; all three runs produced byte-identical results, confirming deterministic behavior. Default Zeek policy scripts were loaded without modification (`local.zeek`), equivalent to evaluating Suricata with the ET Open ruleset without custom tuning.

Table 11: Three-paradigm detection comparison on CTU-13 Neris (DAY 147). Identical hardware, dataset, and corpus for all three systems. Ground truth: 147.32.84.165 (646 malicious flows, 12,075 benign flows). Zeek metrics exclude `CaptureLoss` infrastructure notices (6 entries, not detections). [†] Zeek Precision= 1.0000: every alert correctly identifies the malicious host; low F1 reflects the structural Recall limitation of scripted behavioral detection, not a system deficiency.

System	Paradigm	TP	FP	FN	Prec.	Recall	F1
Suricata 6.0.10	Signature (ET Open)	0	0	646	—	0.0000	0.0000
Zeek 8.1.2 (default)	Scripted behavioral	14	0	632	1.0000 [†]	0.0217	0.0424
aRGus NDR	ML behavioral	646	2	0	0.9969	1.0000	0.9985

Zeek detection detail. Zeek generates 20 raw notices: 6 are `CaptureLoss` infrastructure metadata (excluded from metrics) and 14 are genuine `SSL::Invalid_Server_Cert` detections, all originating from the ground truth IP (147.32.84.165). The bot connected to servers with untrusted certificate chains — Microsoft Update infrastructure (65.55.196.251, 65.55.16.187) and Google services (74.125.224.242) — which Zeek’s TLS dissector flags regardless of whether the connecting host is known to be malicious. This is structural detection: Zeek validates certificate chains as part of protocol analysis, not as a behavioral heuristic.

Table 12: Zeek 8.1.2 behavioral visibility on CTU-13 Neris — observations recorded in structured logs without generating `notice.log` alerts under default policy scripts. Zeek observes the complete behavioral profile of the botnet; the gap between visibility and detection is the core finding of this comparison.

Log	Observation	Count
conn.log	Flows from/to malicious host	31,736
	Unique destination IPs	4,199
	DNS flows	8,896
	HTTP flows (C2 + click fraud)	1,236
	SMTP flows (spam)	63
	SSL flows	63
weird.log	unknown_dce_rpc_auth_type (SMB lateral movement)	33
	bad_HTTP_request (malformed C2 beaconing)	31
	empty_http_request (beaconing)	31
	irc_invalid_command (IRC C2 present)	30
	premature_connection_reuse	28
http.log	Total HTTP requests (GET + POST)	1,377
	Top C2 host: 1.95622.com	300
	Top C2 host: www.1ddwj.com	136
smtp.log	Spam sessions (forged AOL identities)	82
ssl.log	SSL flows from malicious host	63
	Invalid certificates ($\rightarrow$ notices)	47

The three-paradigm spectrum. These results define three structurally distinct detection philosophies:

- **Suricata (signature-based):** requires prior knowledge of the exact threat identity. F1= 0.000 on 15-year-old traffic not because the engine fails, but because no matching rule exists for a retired threat family. Correct behavior of a correct system operating as designed.
- **Zeek (scripted behavioral):** detects structural anomalies with Precision= 1.000 — every alert correctly identifies the malicious host. However, Recall= 0.022 reflects a fundamental design property: default policy scripts alert on specific structural violations (invalid certificates, malformed protocol fields), not on behavioral flow patterns. Table 12 demonstrates that Zeek *observes* the complete behavioral profile of the botnet — IRC commands, HTTP beaconing to 4,199 unique destinations, SMB lateral movement, 82 spam sessions — but does not convert these observations to alerts under default configuration. Zeek is a high-fidelity network observability platform with selective detection capability.
- **aRGus NDR (ML behavioral):** classifies the behavioral footprint of the malware — how many hosts it contacts, what protocols it abuses, the statistical structure of its flows — without requiring prior knowledge of the threat’s identity or any structural protocol violation. Recall= 1.000 and F1= 0.9985 on traffic the classifier had never seen during training.

Scientific significance. This three-way comparison yields a finding that no two-system comparison could produce: *Precision and Recall are not in fundamental tension for ML behavioral detection — they are in structural tension for scripted behavioral detection.* Zeek achieves perfect Precision at the cost of near-zero Recall; aRGus achieves both simultaneously. This is not a deficiency of Zeek: it is the structural consequence of the architectural difference between *detecting structural anomalies* and *classifying behavioral patterns*.

A secondary finding emerges from `weird.log`: the entry `irc_invalid_command: 30` confirms the presence of IRC traffic in the CTU-13 Neris capture, partially refuting the scenario README which describes only HTTP C2. The behavioral reality is more complex — the bot exhibited both IRC and HTTP C2 patterns — reinforcing the argument that any single-modality detector (signature-based or structurally-anomaly-based) is incomplete against multi-protocol botnet behavior [3].

4 Performance Model and Throughput Analysis

5 Limitations

6 Future Work

7 Conclusion

This paper presented ML Defender (aRGus NDR), an open-source network detection and response system built in C++20, designed from first principles to protect the organizations that need protection most and can afford it least.

Evaluated against real botnet traffic from the CTU-13 Neris dataset — traffic the system had never seen during training — ML Defender achieves $F1 = 0.9985$, $Precision = 0.9969$, and $Recall = 1.0000$. The two false positives are fully identified: an mDNS multicast frame and a broadcast frame, both generated by the VirtualBox host-only NIC. Their absence in bare-metal deployments is a reasonable expectation, but has not been empirically verified; bare-metal characterization remains Future Work (§??). The Fast Detector alone produces a 6.61% FPR on purely benign traffic; the ML Detector reduces this to zero real production blocks on the same corpus. End-to-end inference latency ranges from $0.24\ \mu s$ to $1.06\ \mu s$ per flow, entirely in-process, on commodity hardware costing approximately 150–200 USD.

Under progressive load testing, the pipeline processes up to $\sim 34\text{--}38$ Mbps — the ceiling imposed by VirtualBox NIC emulation — with zero packet drops, zero errors, and stable RAM (~ 1.28 GB) across 2.37 million replayed packets. The bottleneck is the hypervisor, not the software.

These results cover one botnet scenario from 2011. They demonstrate architectural viability and effective detection within that scope. The limitations in Section 5 are the honest documentation of a system that does what it claims to do within the boundaries where it has been tested.

A subsequent evaluation (DAY 122) integrated XGBoost 3.2.0 as a hot-swappable plugin and conducted a rigorous temporal holdout study on CIC-IDS-2017. The in-distribution results

(Precision= 0.9945, Recall= 0.9818) confirm that the plugin architecture performs as designed. The out-of-distribution finding — that no threshold satisfies both medical gates on Wednesday data, because application-layer DoS attacks are structurally absent from the training days — is reported not as a failure but as a contribution: a quantified, empirically grounded argument for why production NDR classifiers must be trained on traffic from their actual deployment environment. The Adversarial Capture-Retrain Loop (§??) is the architectural response.

The first empirical comparison of ADR-029 Variant A (eBPF/XDP) and Variant B (libpcap) on x86-64 (DAY 145, §3.12) reveals a hardware-dependent performance inversion: under VirtualBox virtio emulation, libpcap achieves approximately 2× the throughput of eBPF/XDP due to the absence of a native XDP driver path. Both variants confirm functional equivalence at the detection layer. This result provides direct empirical motivation for bare-metal hardware acquisition and establishes the reproducible baseline against which the expected inversion on physical hardware will be measured.

The first direct experimental comparison of aRGus NDR and Suricata 6.0.10 on the same dataset, hardware, and topology (DAY 146, §3.13) yields a result that is both clear and scientifically meaningful: Suricata with 50,010 current ET Open rules generates zero alerts on CTU-13 Neris 2011; aRGus achieves F1= 0.9985 on the same traffic. The zero-alert result is not a failure of Suricata — it is the expected behavior of a signature-based IDS when no matching rule exists for a 15-year-old threat family. The finding provides direct empirical support for the architectural thesis of this paper: *behavioral ML detects what the traffic does; signatures detect what the traffic is known to be*.

The extension to a *three-paradigm comparison* (DAY 147, §3.14) adds Zeek 8.1.2 as the critical intermediate data point: scripted behavioral detection achieves Precision= 1.0000 but Recall= 0.0217, correctly identifying the malicious host in every alert while missing 97.8% of malicious flows. The `weird.log` reveals that Zeek *observes* the complete behavioral profile of the botnet — IRC commands, HTTP beaconing to 4,199 unique destinations, SMB lateral movement, 82 spam sessions — without converting these observations to alerts under default policy scripts. This distinction between *network observability* and *behavioral detection* is the core scientific finding of the three-paradigm comparison. The result could only emerge from a three-way experiment: with two systems, the distinction between “cannot detect” and “chooses not to alert” is invisible.

ML Defender was built because a friend lost his business to ransomware, and because a hospital in Barcelona was paralyzed by an attack on the same day its author was leaving a regional hospital in Extremadura — the hospital that had diagnosed his porphyria and saved his life. The design constraints that followed from those two events were not abstract: open-source, commodity hardware, real-time response. Not a forensic tool. Not a dashboard. A system that acts.

The *Consejo de Sabios* represents a new mode of engineering: one that gives a single independent researcher access to the kind of multi-expert peer review that has historically required institutional affiliation, research groups, and geographic proximity to centers of expertise. The *Test Driven Hardening* methodology that emerged from this collaboration has proven effective against some of the most subtle concurrency and memory bugs in the codebase.

Both contributions — the technical system and the collaborative methodology — are acts of democratization. The parallel is not accidental. It is the point.

For the hospital in Extremadura. For the friend whose business was broken by ransomware. For every small organization that has been told, implicitly or explicitly, that enterprise-grade security is not for them.

It is now.

8 Reproducibility and Artifact Availability

All experiments are reproducible from the public repository: <https://github.com/alonsoir/argus>

Dataset Setup.

```
# CTU-13 Neris (scenario 10) -- required for F1 validation
wget https://mcfp.felk.cvut.cz/publicDatasets/CTU-Malware-Capture-Botnet-42/\
botnet-capture-20110810-neris.pcap -O /vagrant/datasets/ctu13/neris.pcap

# CTU-13 bigFlows -- required for stress test
wget https://mcfp.felk.cvut.cz/publicDatasets/CTU-Malware-Capture-Botnet-42/\
bigFlows.pcap -O /vagrant/datasets/ctu13/bigFlows.pcap
```

F1 Evaluation.

```
make pipeline-stop && make logs-lab-clean && make pipeline-start && sleep 15
make test-replay-neris
python3 scripts/calculate_f1_neris.py \
    /vagrant/logs/lab/sniffer.log --total-events 19135
```

ADR-029 Variant A/B Comparison.

```
# Variant A (eBPF) -- baseline
make test-replay-neris-x86-ebpf
# Logs: /vagrant/logs/lab/tcp replay-ebpf-{10,50,100}mbps.log

# Variant B (libpcap) -- comparison
make test-replay-neris-x86-libpcap
# Logs: /vagrant/logs/lab/tcp replay-libpcap-{10,50,100}mbps.log
```

Suricata Comparative Experiment (DAY 146).

```
# Arrange Suricata experiment VMs (identical hardware spec to aRGus)
make up-suricata

# Execute 3 runs (10 / 50 / 100 Mbps) -- identical topology to aRGus DAY 145
make experiment-suricata-run
```

```
# Parse eve.json -> TP/FP/FN/F1/Recall vs ground truth 147.32.84.165
make experiment-suricata-results
# Output: logs/experiment/suricata_metrics_final.json
# Raw logs: logs/experiment/eve-experiment-suricata-replay-{10,50,100}.json
```

Zeek Three-Paradigm Experiment (DAY 147).

```
# Provision Zeek VM (debian/bookworm64, 8GB RAM, 6 vCPU -- identical to aRGus)
make experiment-zeek-up

# Offline analysis: zeek -r neris.pcap local (x3 runs, deterministic)
make experiment-zeek-run
# Raw logs: logs/experiment/zeek/{10,50,100}mbps/{notice,conn,weird,http,smtp,ssl}.log

# Parse corrected metrics (CaptureLoss excluded)
cd experiments/zeek-comparative && vagrant ssh zeek -c "
  python3 /vagrant/experiments/zeek-comparative/parse_results_zeek_v2.py \
    --logdir /vagrant/logs/experiment/zeek/10mbps \
    --output /vagrant/logs/experiment/zeek/zeek_metrics_v2_10mbps.json"
```

Stress Test.

```
vagrant ssh client -c "sudo tcpreplay -i eth1 --mbps=100 --loop=3 \
  /vagrant/datasets/ctu13/bigFlows.pcap"
```

Determinism. Classifiers embedded in binary; no stochastic inference components; feature extraction deterministic. F1=0.9985 confirmed stable at total_events = 12,563 / 12,605 / 12,723 / 13,930.

Acknowledgments

This work was carried out independently, without institutional funding, without a research group, and without affiliation to any university or laboratory.

The author engaged eight large language models — Claude (Anthropic), Grok (xAI), ChatGPT (OpenAI), DeepSeek, Qwen (Alibaba), Gemini (Google), Kimi (Moonshot AI), and Mistral (Mistral AI) — as structured intellectual collaborators across **147 days** of continuous development. Their contributions to architectural decisions, failure mode identification, debugging, test scenarios, and paper review are documented in the project’s commit history and ADRs. The *Test Driven Hardening* methodology documented in Section ?? emerged directly from this collaboration.

This paper discloses AI involvement openly and precisely. The author remained the final arbiter of all decisions; all empirical validation was performed by the human author.

The author wishes to acknowledge the friend whose business was destroyed by ransomware — who will recognize himself in these pages. And the Hospital Universitario de Badajoz, Extremadura, whose physicians diagnosed a rare disease and gave the author the years needed to build something worth building. Thank you Pedro Risco and Agustín Pijierro.

This work is dedicated to every hospital, school, and small organization that has ever been told that enterprise-grade security is beyond their reach.

References

- [1] Blake Anderson and David McGrew. Identifying encrypted malware traffic with contextual flow data. In *Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 35–46, 2016.
- [2] Anthropic. Project glasswing: Securing critical software for the AI era. <https://www.anthropic.com/glasswing>, April 2026. Accessed: April 2026.
- [3] Hassnain Asad and Ilir Gashi. A perspective–retrospective analysis of diversity in signature-based open-source network intrusion detection systems. *International Journal of Information Security*, 23, 2023. doi: 10.1007/s10207-023-00794-9. Accessed: May 2026.
- [4] Black Fog. The state of ransomware 2025. <https://www.blackfog.com/>, 2025.
- [5] Anna L. Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2): 1153–1176, 2016.
- [6] Sebastian Garcia, Martin Grill, Jan Stiborek, and Alejandro Zunino. An empirical comparison of botnet detection methods. *Computers & Security*, 45:100–123, 2014.
- [7] IBM Security. Cost of a data breach report 2025, 2025.
- [8] INCIBE-CERT. Ciberataque ransomware paraliza actividad del Hospital Clínic de Barcelona. <https://www.incibe.es/>, 2023.
- [9] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Network and Distributed Systems Security Symposium (NDSS)*, 2018.
- [10] Open Information Security Foundation. Suricata open source IDS/IPS/NSM engine. <https://suricata.io/>, 2010.
- [11] André Pinto et al. Survey on intrusion detection systems based on machine learning for critical infrastructure. *Sensors*, 23(5):2415, 2023.
- [12] Martin Roesch. Snort: Lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA)*, pages 229–238, 1999.

- [13] Iman Sharafaldin, Ali Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.
- [14] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *Proceedings of the 2010 IEEE Symposium on Security and Privacy (S&P)*, pages 305–316, 2010. doi: 10.1109/SP.2010.25.